Category A Problems

PROBLEM 1 • Basic → Intermediate

Ticket Hierarchy Foundation & Batch Registration Validator

Problem Statement

CineHub's tech fest needs its ticket system built from the ground up, and registration desks submit attendee IDs in bulk rather than one at a time. Design EventTicket as the shared foundation with a validated constructor, WorkshopTicket as a single-inheritance specialization of it, and a batch validator that processes a whole array of registration attempts at once — silently counting rejections instead of letting one bad ID crash the whole desk's submission.

Requirements

- WorkshopTicket must extend EventTicket directly (single inheritance) and forward the shared fields via super(...) — do not duplicate attendeeId or basePrice as separate fields in WorkshopTicket.
- EventTicket's constructor must reject an attendeeId that's blank, whitespace-only, or shorter than 4 characters.
- registerBatch(...) must attempt to construct one EventTicket per array entry, using try/catch to count rejections — it must not pre-validate the strings itself before construction.

Function Signature(s)

```
public EventTicket(String attendeeId, double basePrice)
public WorkshopTicket(String attendeeId, double basePrice, String track)
void pay(double amount)
double getBalanceDue()
static String registerBatch(String[] attendeeIds, double basePrice)
```

Examples

Input	Output
`new EventTicket("ST1", 500)`	`construction rejected`

Explanation: *"ST1" is only 3 characters — one short of the minimum.*

Input	Output

`WorkshopTicket w = new WorkshopTicket("STU2", 1200, "AI/ML");`
`w.pay(500);`
`w.getBalanceDue()`
	`700.0`

Input	Output
`registerBatch({"STU1", "ST1", "STU2", "  ", "STU3"}, 500)`	`"Registered: 3 | Rejected: 2"`

HINT Let EventTicket's constructor be the only place the validation rule lives — registerBatch should simply try to build one, catch the IllegalArgumentException if construction fails, and count it as a rejection.

CONSTRAINTS basePrice is a positive double. attendeeIds.length up to 500.

PROBLEM 2 • Intermediate

Three Shapes of One Family Tree

Problem Statement

The fest is adding a Premium Workshop tier (extending the regular workshop with a materials kit) and a Hackathon ticket (sharing nothing with workshops beyond the base ticket itself). Build both, write a small classifier that identifies which shape of the family tree a given ticket belongs to, and give the finance desk a way to total up every outstanding balance across the whole mixed family in one pass.

Requirements

- PremiumWorkshopTicket must extend WorkshopTicket (multilevel inheritance, three classes deep); HackathonTicket must extend EventTicket directly, independent of WorkshopTicket entirely (hierarchical inheritance).
- classifyGeneration(...) must decide its answer using instanceof checks alone — do not add a manual "type" field to any class.
- getTotalBalanceDue(...) must accept an array holding any mix of the four ticket types and sum their balances using each ticket's own getBalanceDue() — it must not check each ticket's type before adding it in.

Function Signature(s)

```
public PremiumWorkshopTicket(String attendeeId, double basePrice, String track, double kitFee)
public HackathonTicket(String attendeeId, double basePrice, String teamName)
static String classifyGeneration(EventTicket ticket)
static double getTotalBalanceDue(EventTicket[] tickets)
```

Examples

Input	Output

`new EventTicket("STU1", 500).printTicket()`
`new WorkshopTicket("STU2", 1200, "AI/ML").printTicket()`
`new PremiumWorkshopTicket("STU3", 2000, "Cloud Native", 300).printTicket()`
`new HackathonTicket("STU4", 800, "Byte Force").printTicket()`
	`"Standard Event Ticket | Balance Due: 500.0"`
	`"Workshop Ticket | Track: AI/ML | Balance Due: 1200.0"`
	`"Premium Workshop Ticket | Track: Cloud Native | Kit Fee: 300.0 | Balance Due: 2000.0"`
	`"Hackathon Ticket | Team: Byte Force | Balance Due: 800.0"`

Input	Output
`classifyGeneration(premiumTicket)` `classifyGeneration(hackathonTicket)`	`"Multilevel descendant (3 generations deep)"` `"Hierarchical sibling (independent branch)"`

Input	Output
`getTotalBalanceDue({ standardTicket, workshopTicket, premiumTicket, hackathonTicket })`	`4500.0`

HINT getTotalBalanceDue(...) is a one-line loop body once you trust polymorphism — calling getBalanceDue() on an EventTicket reference already runs each object's own version, regardless of which subclass it actually is.

CONSTRAINTS tickets.length up to 1,000. Every subclass overrides printTicket() to describe itself.

PROBLEM 3 • Intermediate

The Late-Registration Penalty Override & Audit Trail

Problem Statement

Workshops have limited seats, so late registrations should be penalized more heavily than the standard fee — and finance wants an audit trail of every penalty ever applied to a ticket, without any outside code being able to tamper with that history after the fact.

Requirements

- WorkshopTicket must override applyLateFee(double amount) using @Override, calling super.applyLateFee(...) with the amount doubled — it must not reimplement the penalty logic from scratch.
- Every penalty applied (in the base class or any override) must be recorded into a private array field; getLateFeeHistory() must return a defensive copy every time it's called, never the real internal array.

Function Signature(s)

```
protected void applyLateFee(double amount)   // defined in EventTicket, overridden in WorkshopTicket
double[] getLateFeeHistory()
```

Examples

Input	Output

`WorkshopTicket w = new WorkshopTicket(1200);`
`w.pay(1200);`
`w.applyLateFee(100);`
`w.getBalanceDue()`
	`200.0`

Explanation: *1200 paid against a 1200 base leaves 0 due; the workshop's doubled penalty of 200 pushes the balance back up to 200.*

Input	Output

`double[] history = w.getLateFeeHistory();`
`history`
`history[0] = 999;`
`w.getLateFeeHistory()`
	`[200.0]`
	`[200.0]`

Explanation: *The single recorded penalty is the doubled amount (200.0), not the original 100.0 passed in — and tampering with the returned array never reaches the ticket's real internal history.*

HINT super.applyLateFee(amount * 2) reuses the parent's own deduction logic and its own array-recording logic in one call — resist writing a second, separate recording step inside WorkshopTicket.

CONSTRAINTS Up to 10 late fees per ticket. amount is always a positive double when applyLateFee is called.

PROBLEM 4 • Intermediate → Advanced

The Nightly Ticket Announcer

Problem Statement

At the end of each day, the fest's announcement board reads out every ticket sold — mixing standard tickets and workshop tickets in one list, and building the announcement as a single assembled report rather than many separate print statements — while separately reaching a workshop-specific detail that a plain EventTicket reference can't see on its own.

Requirements

- batchPrint(...) must loop over an EventTicket[] and call printTicket() polymorphically — no instanceof-based if-else chain deciding what to print.
- The final report must be assembled with StringBuilder, not repeated String concatenation inside the loop.
- For any ticket that is genuinely a WorkshopTicket, use instanceof to guard a downcast that reaches its track — attempting the same downcast on a ticket that isn't a WorkshopTicket must be avoided, not attempted and caught.

Function Signature(s)

```
static String batchPrint(EventTicket[] tickets)
```

Examples

Input	Output
`batchPrint({ new EventTicket(500), new WorkshopTicket(1200, "AI/ML") })`	`"Standard | Balance: 500.0 | Workshop | Track: AI/ML | Balance: 1200.0 [Track via downcast: AI/ML] | "`

Input	Output
`EventTicket plain = new EventTicket(500);` `WorkshopTicket bad = (WorkshopTicket) plain;`	`ClassCastException at runtime`

Explanation: *This compiles fine — the compiler only checks that WorkshopTicket is some kind of EventTicket — but plain is genuinely not a WorkshopTicket at runtime, so the cast fails the moment it executes.*

HINT The instanceof check and the downcast go together as a pair — check first, then cast, and only inside the branch where the check already passed. Append to the same StringBuilder across every loop iteration, don't create a new one per ticket.

CONSTRAINTS The array may contain any mix of EventTicket and its subclasses, but never null in this problem (Problem 5 adds that).

PROBLEM 5 • Advanced

Fest-Wide Ticket Issuance, Promo Codes & Nightly Settlement Engine

Problem Statement

The fest wants one system to do four things at once: guarantee every ticket gets a unique, auto-incrementing ID that can never be reassigned once set; validate a promo code's format using nothing but character-by-character string checks; offer both a flat and a mode-aware way to pay for a ticket; and reconcile a night's worth of tickets — including a group-booking variant — without ever crashing on a missing entry.

Requirements

- Every EventTicket must receive a ticketId from a shared static counter, incremented once per object, and stored in a final field — never settable from outside and never reassignable after construction.
- isValidPromoCode(...) must check the exact format "F" + three digits + one uppercase letter (e.g. "F123A") using charAt() and Character.isDigit()/isUpperCase() — not a regular expression.
- Provide two overloaded pay(...) methods: a flat-amount version, and a version that also records a payment mode, reusing the flat version internally rather than duplicating its logic.
- processNightlySettlement(...) must use instanceof to separate GroupTicket from a regular EventTicket, and must never throw on a null batch entry.

Function Signature(s)

```
public EventTicket(double basePrice)   // assigns a final ticketId internally
void pay(double amount)
void pay(double amount, String mode)
static boolean isValidPromoCode(String code)
static int getTicketsIssued()
public GroupTicket(double basePrice, int groupSize)
static String processNightlySettlement(EventTicket[] tickets)
```

Examples

Input	Output
EventTicket t1 = new EventTicket(500); t1.ticketId EventTicket.getTicketsIssued()	"TCK-1001" 1

Input	Output
EventTicket.isValidPromoCode("F123A") EventTicket.isValidPromoCode("F12A") EventTicket.isValidPromoCode("X123A")	true false false

Explanation: *"F12A" is one character too short; "X123A" has the right shape but the wrong leading letter.*

Input	Output

t1.pay(200);
t1.pay(200, "UPI");
t1.getBalanceDue()
	100.0

Explanation: *Both payments land on the same underlying balance — the two-argument overload only adds a printed mode before delegating to the one-argument version.*

Input	Output
`processNightlySettlement({ new GroupTicket(2000, 5), null, new EventTicket(500) })`	`"2 processed | 1 null skipped | 1 group | 1 individual"`

HINT Increment the static counter inside the constructor itself, before building the final ticketId string from its new value. For the promo code, check the length first — a wrong length should fail immediately, before you ever call charAt() on a position that might not exist.

CONSTRAINTS Batch size up to 5,000. groupSize is a positive integer.